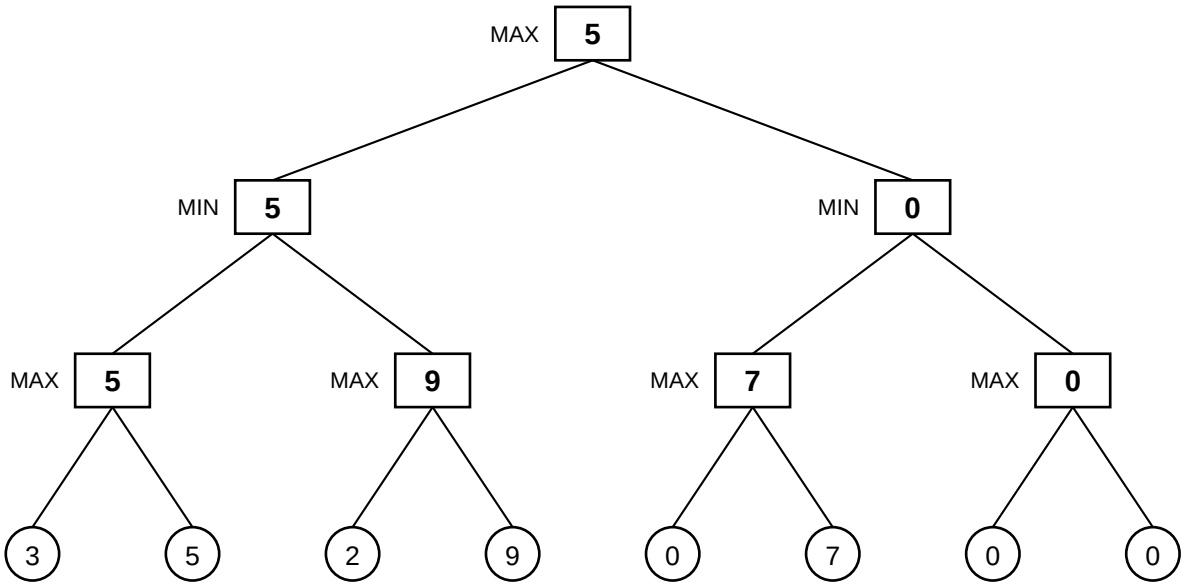
Name: M. Umair Roll No: 048
Lab Task No: 5

Question 1: Minimax Hand Trace

Leaf values: [3, 5, 2, 9, 0, 7, 0, 0] in a complete binary tree. The root is a MAX node, the next level is MIN, then MAX again, then the leaves.



Steps:
Bottom MAX level: max(3,5) = 5, max(2,9) = 9, max(0,7) = 7, max(0,0) = 0
MIN level: min(5,9) = 5, min(7,0) = 0
Root MAX: max(5,0) = **5**

Question 2: Leaf Evaluation Counter

Plain minimax looks at all 8 leaves. Alpha-beta pruning can skip leaves that cannot change the answer.

```
def minimax(scores, depth, index, max_turn, count):
    if depth == 3:
        count[0] += 1
        return scores[index]
    if max_turn:
        return max(
            minimax(scores, depth + 1, index * 2, False, count),
            minimax(scores, depth + 1, index * 2 + 1, False, count))
    else:
        return min(
            minimax(scores, depth + 1, index * 2, True, count),
            minimax(scores, depth + 1, index * 2 + 1, True, count))


def alphabeta(scores, depth, index, max_turn, alpha, beta, count):
    if depth == 3:
        count[0] += 1
        return scores[index]
    if max_turn:
        value = float('-inf')
        for i in range(2):
            value = max(value, alphabeta(scores, depth + 1, index * 2 + i,
```

```
                                    False, alpha, beta, count))
            alpha = max(alpha, value)
            if alpha >= beta:
                break
        return value
    else:
        value = float('inf')
        for i in range(2):
            value = min(value, alphabeta(scores, depth + 1, index * 2 + i,
                                    True, alpha, beta, count))
            beta = min(beta, value)
            if alpha >= beta:
                break
        return value


scores = [3, 5, 2, 9, 0, 7, 0, 0]
m_count = [0]
a_count = [0]
m = minimax(scores, 0, 0, True, m_count)
a = alphabeta(scores, 0, 0, True, float('-inf'), float('inf'), a_count)
print("Minimax:", m)
print("Minimax Leaves:", m_count[0])
print("Alpha-Beta:", a)
print("Alpha-Beta Leaves:", a_count[0])

new_scores = [3, 5, 2, 9, 0, 0, 0, 7]
new_count = [0]
new_value = alphabeta(new_scores, 0, 0, True, float('-inf'), float('inf'), new_count)
print("Reordered Value:", new_value)
print("Reordered Leaves:", new_count[0])
```

Question 3: Leaf Reordering

Original: [3, 5, 2, 9, 0, 7, 0, 0]

Reordered: [3, 5, 2, 9, 0, 0, 0, 7]

Results:

Test	Value	Leaves checked
Minimax	5	8
Alpha-Beta (original)	5	8
Alpha-Beta (reordered)	5	6

The answer stays 5 every time. With the reordered leaves, the right MIN branch finds a 0 early, which is already worse than the 5 MAX has, so the last two leaves (0 and 7) are skipped. Good leaf order means more pruning.

GitHub Link

https://github.com/umair-900/Ai-lab-Task